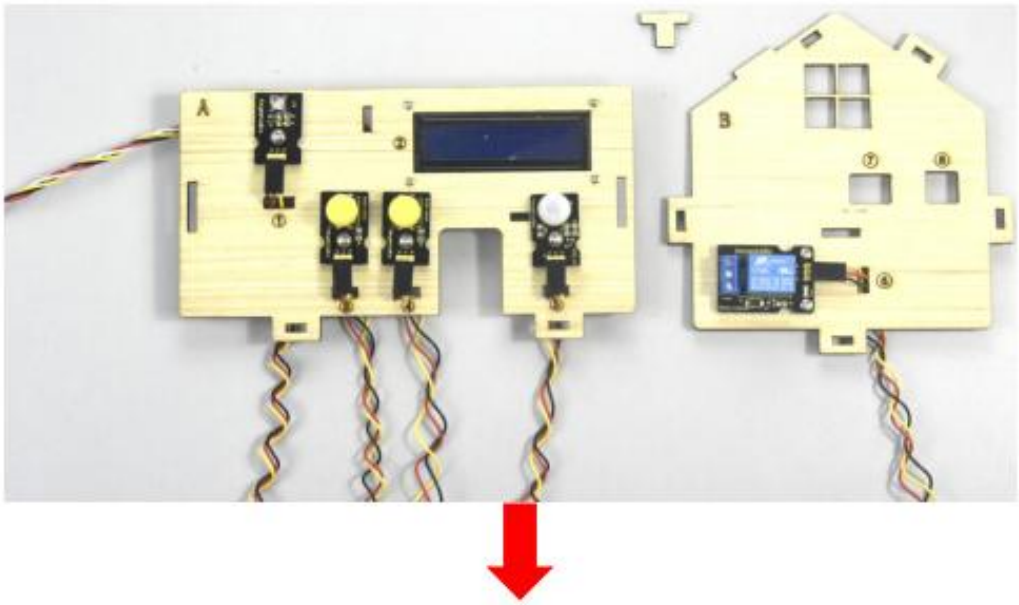
# Smart Home Kit for Arduino Lesson Guide



# How to Work Through Each Lesson

Follow the same sequence each time. Once you have completed the steps and passed the check, you can move on to the challenges!

1. First, look at the circuit diagram and note which components and pins you need.
2. Disconnect the USB cable before changing wires. Connect G, V, and S as shown in the diagram, and check each connection.
3. Connect the USB cable, select the Keyestudio / Arduino Uno board and the correct port, then click Upload.
4. Try the challenge by changing only one idea at a time. Note what you changed and what you observed.

CONNECTIONS G = ground (GND) • V = power (5V) • S = signal. On the sensor, always follow the letters shown in the diagram. Do not change connections while the board is powered.

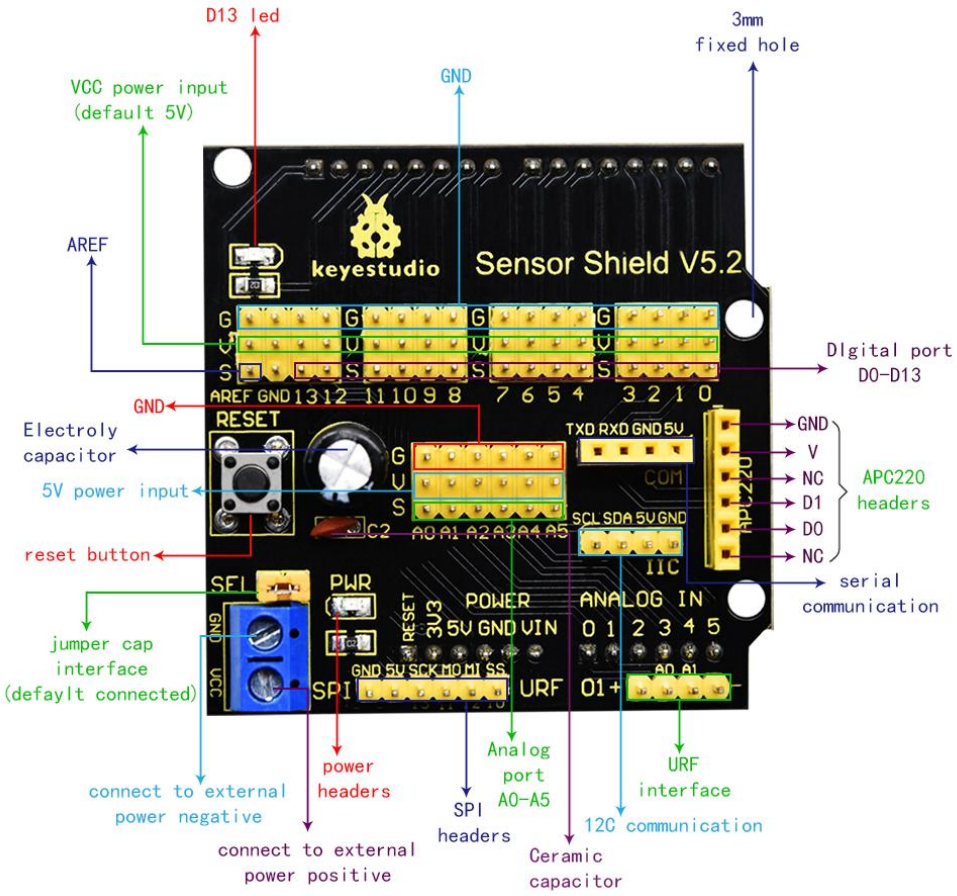
### Quick Check Before You Click Upload

- The USB cable was disconnected while I was building the circuit.
- G, V, and S match the positions in the diagram.
- The correct port and board are selected.
- I saved a backup copy of the file before changing the code.

### Safety Rules

- Use the relay only with the kit's low-voltage circuit. Never connect it to a wall outlet or household mains electricity.
- The MQ-2 is a sensor for a classroom experiment, not an alarm. Do not test it with a flame, lighter, or household gas.
- Keep water and moisture away from the board. The steam sensor is not waterproof.
- Keep fingers, hair, and loose objects away from the fan. Do not force the servo by hand.

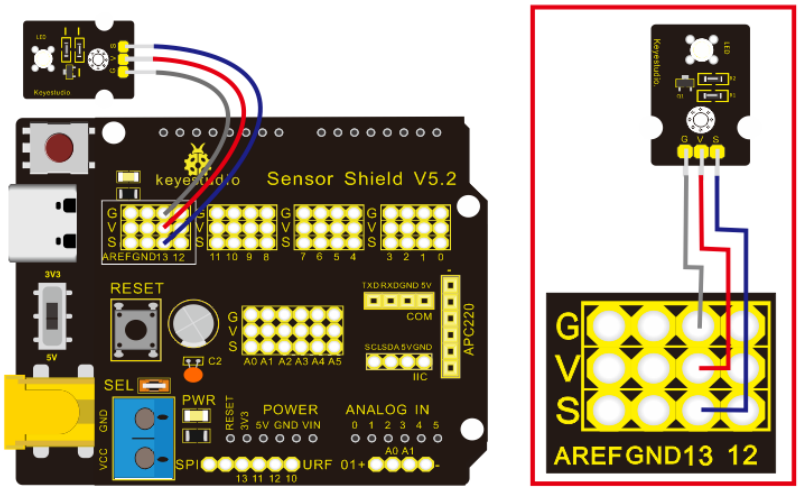
The kit guide contains the full description of each project. The page numbers listed on the worksheets refer to the KS0085 PDF.


D13 led
3mm
fixed hole
GND
VCC power input
(default 5V)
AREF
keyestudio
Sensor Shield V5.2
Dlgital port
D0-D13
AREF GND 13 12 11 10 9 8 7 6 5 4 3 2 1 0
GND
RESET
Electroly
capacitor
5V power input
reset button
TXD RXD GND 5V
COM
SCL SDA 5V GND
IIC
A0 A1 A2 A3 A4 A5
GND
V
NC
D1
D0
NC
APC220
APC220
headers
serial
communication
SEL
PWR
POWER
ANALOG IN
RESET 3V3 5V GND VIN
0 1 2 3 4 5
jumper cap
interface
(defaylt connected)
GND 5V SCK MO MI SS
SPI
URF
O1+
A0 A1
connect to external
power negative
power
headers
SPI
headers
Analog
port
A0-A5
URF
interface
12C communication
connect to external
power positive
Ceramic
capacitor

Goal: Connect the LED module to G, V, and D13.

### Wiring

Connect the LED module to G, V, and D13.




Wiring diagram for the white LED on output D13.
On 3-pin cables: G = ground • V = 5V • S = signal

### Programming

To control the LED, we will use the digitalWrite function.
In Arduino programming, digitalWrite() sends a digital signal to a pin by setting its voltage to either HIGH (high voltage, usually 5V or 3.3V) or LOW (zero voltage, 0V / ground).

```
digitalWrite(pin, value);
```

- pin: The number of the digital pin you want to control (e.g., 13, A0).
- value: The value you want to set. It can only be one of two values: HIGH (on / power supplied) or LOW (off / no power).

### Basic Rules and Operation

- Prerequisite (pinMode): Before using digitalWrite(), you must declare the pin as an output in setup() using pinMode(pin, OUTPUT).
- What HIGH does: It applies the board's maximum voltage (5V on most Arduino boards such as the Uno, or 3.3V on newer boards / ESP32) to turn on an LED or activate a relay.
- What LOW does: It sets the voltage to zero (0V), turning off the connected device.

### Sample Program

```
```

```
const int LED_PIN = 13;
void setup() {
  pinMode(LED_PIN, OUTPUT);
}
void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(1000);
  digitalWrite(LED_PIN, LOW);
  delay(1000);
}
```

Check

Click Upload and make sure the LED blinks once every second as expected. If it does not work, check:

- That the code shows “Verify and Upload: Success”.
- That the COM / ttyUSB port is correct.
- That the connections are correct.

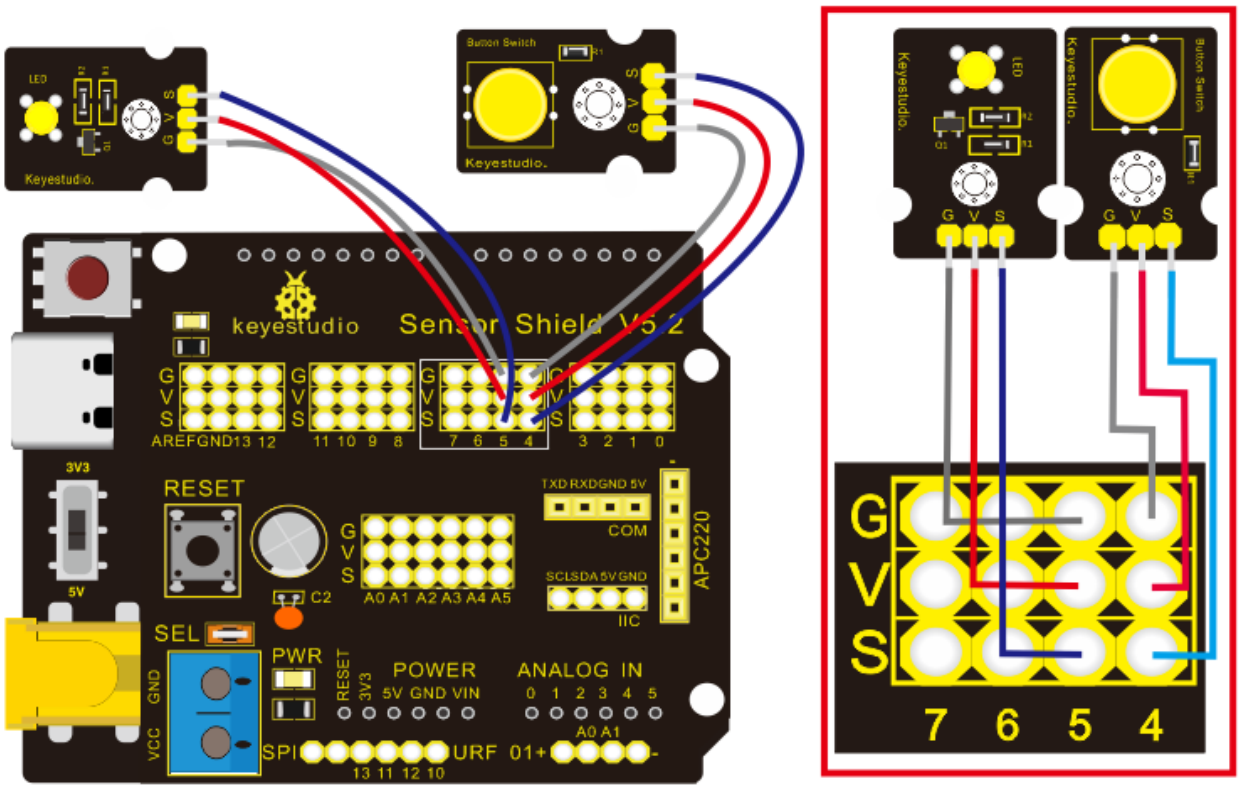
Challenges

1. Change the LED blink rate so that it is no longer visible to our eyes.
2. Add a second LED. When the first LED turns on, the second one should turn off.
3. Super challenge! Create a sequence of blinks with two LEDs. Your classmates should watch and copy it! (And you should be able to copy theirs, too.)

## Lesson 02 - Button

### Wiring

Connect the button module to G, V, and D4, and the LED to D5, as shown in the diagram.




### Programming

To check the button's state, we will use the "opposite" of digitalWrite: digitalRead. The digitalRead() function reads the state of a digital pin, that is, whether voltage is present (HIGH) or not (LOW).

```
buttonState = digitalRead(pin);
```

- pin (input parameter): The number of the digital pin you want to check. In this example, use 4 (or the buttonPin variable).

- Return value: The function returns one of two possible states, depending on the voltage it detects on the pin at that moment:
    - HIGH: The pin has a high voltage (5V or 3.3V). This usually means the button is pressed.
    - LOW: The pin has no voltage (0V / ground). This means the button is not pressed.

Basic Rules and Operation

- Prerequisite (pinMode): Before reading the pin, declare it as an input in setup() using pinMode(4, INPUT).
- Storing the value: Because digitalRead() returns a value (HIGH or LOW), we store it in a variable (e.g., buttonState) so we can check it later with an if statement.

**Sample Program 1**

```
const int buttonPin = 4; // The button is on Pin 4
int buttonState = 0;    // Variable to store HIGH or LOW

void setup() {
 pinMode(buttonPin, INPUT);  // Set Pin 4 as an input (reads)
 Serial.begin(9600);       // Start communication with the computer
}

void loop() {
 // Read Pin 4 and store the result (HIGH or LOW)
 buttonState = digitalRead(buttonPin);

 // Display the value on the computer screen
 Serial.print("Button State: ");
 Serial.println(buttonState);

 delay(100); // Short delay so we have time to read the value
}
```

**Check:**

1. Click Upload in the Arduino IDE.
2. Make sure the COM / ttyUSB port and board are selected correctly.
3. Open the Serial Monitor (the magnifying-glass icon at the top right of the Arduino IDE).
4. What you should see: When you are not pressing the button, it should display 0 (LOW). When you press and hold it, it should immediately change to 1 (HIGH).

Now that we know the button works perfectly 😊, we will also connect the LED to Pin 5 so the input (button) can control the output (LED).

### Sample Program 2

```
const int buttonPin = 4; // The button is on Pin 4
const int ledPin = 5;    // The LED is on Pin 5

int buttonState = 0;     // Variable to store HIGH or LOW

void setup() {
  pinMode(buttonPin, INPUT);  // Set Pin 4 as an input (reads)
  pinMode(ledPin, OUTPUT);    // Set Pin 5 as an output (writes)
}

void loop() {
  // 1. Read Pin 4 and store the result (HIGH or LOW)
  buttonState = digitalRead(buttonPin);

  // 2. Check which value digitalRead returned
  if (buttonState == HIGH) {
    digitalWrite(ledPin, HIGH); // If it reads HIGH, turn on the LED
  } else {
    digitalWrite(ledPin, LOW);  // If it reads LOW, turn off the LED
  }
}
```

Check

Click Upload and make sure the LED turns on when you press the button and turns off when you release it:

- That the code shows “Verify and Upload: Success”.
- That the COM / ttyUSB port is correct.
- That the connections are correct.

Challenges

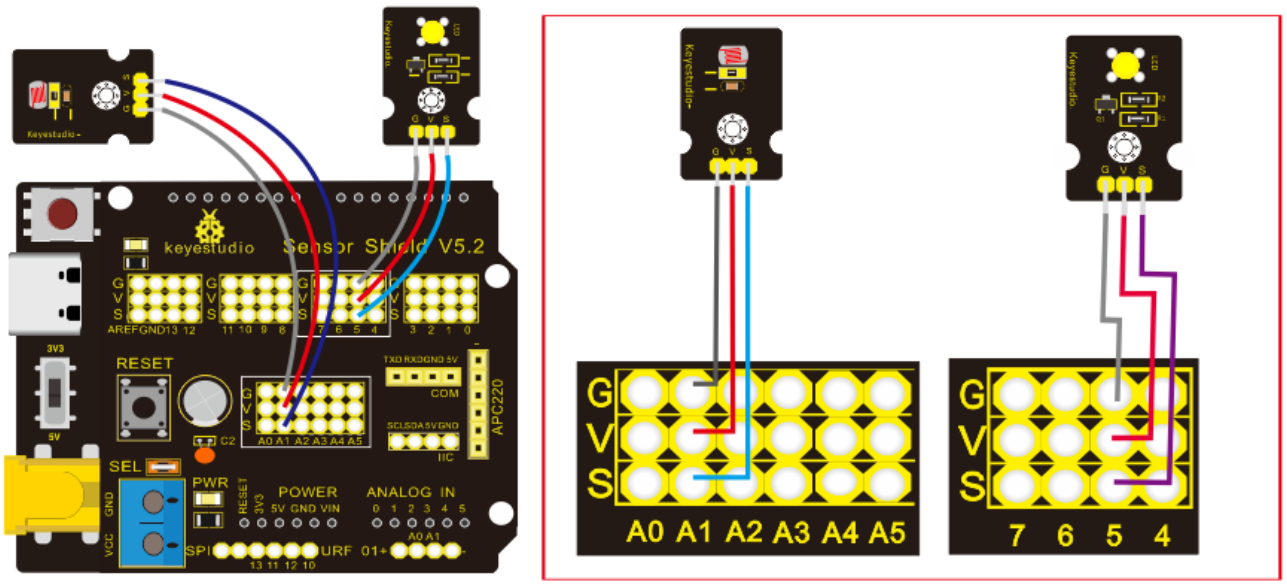
1. While you hold down the button, the LED should not simply stay on; it should blink quickly (like a car’s turn signal). When you release the button, it should turn off immediately.
2. Make the LED work in reverse! When you are not pressing the button, the LED should be on. When you press the button, it should turn off.
3. So far, the LED turns off as soon as you release the button. Now make the button work like a room light switch: press and release once to turn the LED on and keep it on; press and release a second time to turn it off. (Hint: You will also need a short delay(200) so the Arduino is not confused by button bounce.)
4. **When the program starts, the LED is off. The Arduino waits for a random amount of time (e.g., 2 to 5 seconds). Suddenly, the LED turns on!
   You must press the button as quickly as possible. If you press it within 300 milliseconds, the LED blinks happily 3 times (You win!). If you are too slow, the LED fades out slowly (You lose!).**

# Lesson 03 - Measuring Light

### Wiring

Connect the LDR (light-dependent resistor) module to G, V, and A0, and the LED to D5, as shown in the diagram.

An LDR is a resistor whose value changes with light! You can use a multimeter to measure exactly how it changes with the amount of light.




### Programming

### To measure light intensity, we will use a new function called analogRead.

The analogRead() function reads the state of an analog pin. It converts the voltage sent by the LDR module into a digital number from 0 to 1023.

```
ldrValue = analogRead(pin);
```

- pin (input parameter): The number of the analog pin you want to check. In our example, use A0 (or the ldrPin variable).
- Return value: The function returns an integer from 0 to 1023, depending on the light reaching the module at that moment:
    - High value (near 1023): Corresponds to high voltage (5V). This means the room is very bright.
    - Low value (near 0): Corresponds to low voltage (0V). This means it is dark.

### Basic Rules and Operation

- Analog pins: Analog pins (A0-A5) work automatically as inputs, so you do not need to write pinMode(A0, INPUT) in setup().
- Storing the value: Because analogRead() returns a changing value (0-1023), we store it in a variable (e.g., ldrValue) so we can display it or check it later with an if statement.

### Sample Program 1

```
const int ldrPin = A0;  // The midpoint of the voltage divider on analog pin A0
int ldrValue = 0;      // Integer variable to store the light level
```

```
void setup() {
 Serial.begin(9600);  // Start communication with the computer
}

void loop() {
 // Read the analog value from the breadboard (from 0 to 1023)
 ldrValue = analogRead(ldrPin);

 // Display the value on the computer screen
 Serial.print("LDR Value: ");
 Serial.println(ldrValue);

 delay(100); // Short delay so we have time to read the value
}
```

**Check:**

1. Click Upload in the Arduino IDE.
2. Make sure the COM / ttyUSB port and board are selected correctly.
3. Open the Serial Monitor (the magnifying-glass icon at the top right of the Arduino IDE).
4. What you should see: When you shine light on it, the number should rise toward 1023. When you cover it with your hand (darkness), the number should fall toward 0.
5. Take a note: See what value appears when you cover it completely (e.g., 280). This will be your "darkness threshold."

**Sample Program 2**

```
const int ldrPin = A0;  // The LDR is on Pin A0
const int ledPin = 5;   // The LED is on Pin 5
int ldrValue = 0;      // Integer variable to store the light level

void setup() {
 pinMode(ledPin, OUTPUT);    // Set Pin 5 as an output (writes)
}
void loop() {
 // 1. Read Pin A0 and store the result (0 to 1023)
 ldrValue = analogRead(ldrPin);

 // 2. Check whether the value fell below 300 (meaning it got dark)
 if (ldrValue < 300) {
  digitalWrite(ledPin, HIGH); // If it got dark, turn on the LED
 } else {
  digitalWrite(ledPin, LOW);  // If it is bright, turn off the LED
 }
}
```

**Check**

Click Upload and make sure the LED turns on automatically when you cover the LDR with your hand and turns off when you uncover it:

1. That the code shows "Verify and Upload: Success".
2. That the COM / ttyUSB port is correct.

3. That the connections are correct.
4. Depending on the room lighting, you may need to change the darkness threshold!

### Challenges!

1. In the example, the LED turns on immediately. Now make it turn on only if darkness lasts continuously for more than 2 seconds (so it does not turn on unnecessarily if a brief shadow or a bird simply passes in front of the sensor).
2. Build a refrigerator alarm! When the LDR is dark (door closed), the LED is off. If someone opens the door and light enters, the Arduino should wait 5 seconds (time to take something out). If the door stays open longer, the LED should flash wildly to warn that it was left open! This will be even more interesting in the next lesson, when we add a buzzer!

# Lesson 04 - Sound!

### Wiring

Connect the pins of the Passive Buzzer module directly to the board using jumper wires:

- Connect pin G (GND / ground) to G (GND) on the Arduino.
- Connect pin V (VCC / power) to V (5V) on the Arduino.
- Connect pin S (signal) to digital Pin 3 on the Arduino.

The Passive Buzzer Module contains the component mounted on a small board.
Unlike an active buzzer, a passive buzzer works like a tiny speaker.

- If we simply supply steady power (HIGH), it will make no sound (perhaps only a faint “click”).
- To make a sound, we must send it a signal that switches at a specific frequency (pulses), causing its internal membrane to vibrate.
- Its main advantage is that changing the frequency changes the note! This lets it play different musical notes and melodies.

To understand how the Arduino produces sounds with the buzzer, or blinks an LED so fast that it “tricks” our eyes, we need to know two basic concepts: period and frequency.

- Period (T): The total time required to complete one full cycle of a repeating phenomenon (e.g., the LED turns on AND off). It is measured in seconds (s).
- Frequency (f): The number of times this cycle repeats in 1 second. It is measured in Hertz (Hz).

Their mathematical relationship is as follows:

$$frequency = \frac{1}{period} \rightarrow f = \frac{1}{T}$$

In the basic code, the LED blinks once per second:

```
digitalWrite(ledPin, HIGH); delay(1000);
digitalWrite(ledPin, LOW);  delay(1000);
```

Question
What do you think the period is?
What is the frequency?

T = .... s  f = .... Hz

The Biological Limit and Human Vision
When we reduced delay() in the code, the LED blinked faster and faster.

- If we set delay(10) (that is, 10 ms on and 10 ms off), the total period is 10 + 10 = 20 ms.
- The frequency then becomes:
- That means the LED blinks 50 times per second!

The “Trick” Our Eyes Play (Flicker Fusion):
The human visual system has a biological speed limit. When a light blinks with a period shorter than 12 to 15 ms (equivalent to frequencies above about 70 to 85 Hz), the brain cannot register the brief off period.

This phenomenon is called flicker fusion. To an observer, the LED stops flickering and appears to stay on continuously (just at slightly lower brightness).
Modern technology works through this same “trick”:

- Mobile phone and PC screens: They operate at 60 Hz, 90 Hz, or 120 Hz, refreshing the image so quickly that graphics look perfectly steady and motion appears smooth.
- Cinema: Films show 24 still images (frames) per second, but a mechanism flashes light over each frame two or three times (raising the frequency to 48 Hz or 72 Hz), so we see continuous motion without tiring our eyes.

With a passive buzzer, instead of tricking the eye by blinking an LED, we make the air move! When we write tone(buzzerPin, 440); (which corresponds to the musical note A), the Arduino switches Pin 3 at a frequency of 440 times per second. This means the vibration period is only 2.2 ms.
This extremely fast electrical switching makes the buzzer’s internal membrane vibrate. The vibration travels through the air, producing sound waves that our ears interpret as a clear musical tone.

Answer

- Period: One full cycle (on + off) lasts 1000 ms + 1000 ms = 2000 ms (that is, 2 seconds). Therefore, T = 2 s.
- Frequency: In 1 second, only half of the cycle can occur. Dividing gives a frequency of 0.5 Hz.

### Programming: Sound Functions

To control notes, sound frequency, and duration on the Passive Buzzer Module, Arduino provides two ready-to-use, powerful functions:
A) The tone() function
This function creates a square-wave signal (frequency) on the output pin, causing the buzzer membrane to vibrate and produce a specific musical note.

```
tone(pin, frequency);
// or
tone(pin, frequency, duration);
```

- pin: The Arduino digital pin that sends the signal to the module (in this lesson, pin 3).
- frequency: The sound frequency in Hertz (Hz). The lower the number, the lower the pitch. The higher the number, the higher / sharper the pitch (e.g., 262 is the note C, and 440 is the note A).
- duration (optional): The duration of the sound in milliseconds.

B) The noTone() function
Immediately stops sound generation on the specified pin. If we play a note with tone() without specifying a duration, the buzzer will keep sounding until we call noTone().

```
noTone(pin);
```

Sample Program 1

This code continuously changes the frequency sent to the module (on Pin 3), creating an alternating sound like an ambulance siren.

```
const int buzzerPin = 3; // Passive Buzzer signal on Pin 3

void setup() {
  pinMode(buzzerPin, OUTPUT); // Set Pin 3 as an output
}

void loop() {
  tone(buzzerPin, 440); // Play 440 Hz (note A)
  delay(500);           // Keep the sound on for half a second
  tone(buzzerPin, 880); // Play 880 Hz (one octave higher)
  delay(500);           // Keep the sound on for half a second
}
```

**Check:**

1. Click Upload in the Arduino IDE.
2. Make sure the COM / ttyUSB port is selected correctly.
3. What you should hear: The buzzer should automatically alternate its sound every half second without stopping.

### Sample Program 2: Sound with a Pause (BeepBlink or BlinkBeep)

This code produces a short beep, then uses noTone() to create a clear one-second silence.

```
const int buzzerPin = 3; // Passive Buzzer signal on Pin 3

void setup() {
  pinMode(buzzerPin, OUTPUT); // Set Pin 3 as an output
}

void loop() {
  tone(buzzerPin, 1000); // Play a clear 1000Hz tone
  delay(150);            // Keep the tone on for 150ms
  noTone(buzzerPin);     // Stop the sound completely
  delay(1000);           // Wait 1 second in silence before the next beep
}
```

**Check:**

1. Click Upload in the Arduino IDE.
2. What you should hear: The buzzer should make a short beep every second, like an alarm clock.

Challenges:

- Modify the code so the buzzer does not just make a continuous beep, but sends the international SOS distress signal in Morse code. SOS consists of 3 short beeps, 3 long beeps, and 3 short beeps.
- Create the sound effect of a countdown! The buzzer should make short beeps with the gaps between them getting shorter and shorter (e.g., the delay between beeps starts at 1000 ms and gradually decreases to 0, at which point a long, continuous “explosion” sound plays).

Challenge 3: “Theremin - Music with Shadows” (Boss Level)
Goal: Combine the LDR Module (Pin A0) from the previous lesson with the Buzzer Module (Pin 3).
Build a futuristic musical instrument that changes its sound without being touched! As you bring your hand closer to the LDR and block the light, the buzzer’s pitch should change automatically (becoming lower or higher).
• Hint: Use analogRead(A0) and map() to convert the light values (0-1023) directly into audible frequencies (e.g., 150 to 1500 Hz) for tone().

### Theremin Solution

```
const int ldrPin = A0;      // The LDR module on analog pin A0
const int buzzerPin = 3;    // Passive Buzzer module on Pin 3

int ldrValue = 0;           // Variable to store the light value
int frequency = 0;          // Variable to store the sound frequency

void setup() {
  pinMode(buzzerPin, OUTPUT); // Set Pin 3 as an output
  Serial.begin(9600);         // Start Serial Monitor for checking
}

void loop() {
  // 1. Read the analog light value (0 to 1023)
  ldrValue = analogRead(ldrPin);

  // 2. KEY FUNCTION: Convert the LDR range (0-1023)
  // into an audible frequency for the buzzer (e.g. 150-1500Hz)
  frequency = map(ldrValue, 0, 1023, 150, 1500);

  // 3. Send the frequency to the buzzer to produce sound
  tone(buzzerPin, frequency);

  // Print values on the computer for checking and understanding
  Serial.print("Light (LDR): ");
  Serial.println(ldrValue);
  Serial.print(" -> Frequency (Hz): ");
  Serial.println(frequency);

  delay(10); // Very short delay for an immediate, smooth sound response
}
```

# Lesson 4.5 - Making Music

### Challenge 1: “Build a Piano!”

Goal: Connect 3 button modules (for the notes C, D, and E) and the Passive Buzzer module. When you press a button, the buzzer should play the corresponding frequency from the table (C=262 Hz, D=294 Hz, E=330 Hz). When you release it, the sound should stop.

```
#define NOTE_C 262
#define NOTE_D 294
#define NOTE_E 330
..
..
..
const int buzzerPin = 3;
const int btnC = 4;
const int btnD = 6;
const int btnE = 7;

void setup() {
  pinMode(buzzerPin, OUTPUT);
  pinMode(btnC, INPUT);
  pinMode(btnD, INPUT);
  pinMode(btnE, INPUT);
}

void loop() {
  // Check whether the first button (C) was pressed
  if (digitalRead(btnC) == HIGH) {
    tone(buzzerPin, NOTE_C);
  }
...
  else {
    noTone(buzzerPin);
  }
```

```
}
```

### Challenge 2: “Build a Synthesizer!”

NOTE FREQUENCY CHART | HEROIC AUDIO

| | Octave 0 | Octave 1 | Octave 2 | Octave 3 | Octave 4 | Octave 5 | Octave 6 | Octave 7 | Octave 8 | Octave 9 | Octave 10 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| C | 16.35 | 32.70 | 65.41 | 130.81 | 261.63 | 523.25 | 1046.50 | 2093.00 | 4186.01 | 8372.02 | 16744.04 |
| C# | 17.32 | 34.65 | 69.30 | 138.59 | 277.18 | 554.37 | 1108.73 | 2217.46 | 4434.92 | 8869.84 | 17739.69 |
| D | 18.35 | 36.71 | 73.42 | 146.83 | 293.66 | 587.33 | 1174.66 | 2349.32 | 4698.64 | 9397.27 | 18794.55 |
| D# | 19.45 | 38.89 | 77.78 | 155.56 | 311.13 | 622.25 | 1244.51 | 2489.02 | 4978.03 | 9956.06 | 19912.13 |
| E | 20.60 | 41.20 | 82.41 | 164.81 | 329.63 | 659.26 | 1318.51 | 2637.02 | 5274.04 | 10548.08 | |
| F | 21.83 | 43.65 | 87.31 | 174.61 | 349.23 | 698.46 | 1396.91 | 2793.83 | 5587.65 | 11175.30 | |
| F# | 23.12 | 46.25 | 92.50 | 185.00 | 369.99 | 739.99 | 1479.98 | 2959.96 | 5919.91 | 11839.82 | |
| G | 24.50 | 49.00 | 98.00 | 196.00 | 392.00 | 783.99 | 1567.98 | 3135.96 | 6271.93 | 12543.86 | |
| G# | 25.96 | 51.91 | 103.83 | 207.65 | 415.30 | 830.61 | 1661.22 | 3322.44 | 6644.88 | 13289.75 | |
| A | 27.50 | 55.00 | 110.00 | 220.00 | 440.00 | 880.00 | 1760.00 | 3520.00 | 7040.00 | 14080.00 | |
| A# | 29.14 | 58.27 | 116.54 | 233.08 | 466.16 | 932.33 | 1864.66 | 3729.31 | 7458.62 | 14917.24 | |
| B | 30.87 | 61.74 | 123.47 | 246.94 | 493.88 | 987.77 | 1975.53 | 3951.07 | 7902.13 | 15804.26 | |

Goal: Instead of fixed notes, a synthesizer creates unusual sounds using loops (for). Program 2 buttons (Pins 4 and 6): the first button should make the frequency drop rapidly (Laser effect), and the second should make it rise and fall quickly (Siren effect).

```
const int buzzerPin = 3;
const int btnLaser = 4;
const int btnSiren = 6;

void setup() {
  pinMode(buzzerPin, OUTPUT);
  pinMode(btnLaser, INPUT);
  pinMode(btnSiren, INPUT);
}

void loop() {
  // Button 1: Laser effect (frequency falls from 1000 to 200)
  if (digitalRead(btnLaser) == HIGH) {
    for (int freq = 1000; freq > 200; freq -= 10) {
      tone(buzzerPin, freq);
      delay(2);
    }
    noTone(buzzerPin);
  }
  // Button 2: Siren effect (complete the for loop to raise the frequency)
  else if (digitalRead(btnSiren) == HIGH) {
    // Create a for loop that starts at 500 Hz and rises to 1200 Hz

  }
  else {
    noTone(buzzerPin);
  }
}
```

### Challenge 3

Experiment with the contents of the repository below.

https://github.com/robsoncouto/arduino-songs